Regression_Assignment_Analysis

1. Mutiple Linear Regression

```
[10]: # spitting the training and test inputs
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.30,random_state=0)
```

```
[11]: from sklearn.linear_model import LinearRegression
regressor=LinearRegression()
```

```
[12]: regressor.fit(X_train,y_train)
```

```
[12]: ▾ LinearRegression
      ▶ Parameters
```

```
[13]: y_pred= regressor.predict(X_test)
```

```
[14]: from sklearn.metrics import r2_score
r_score = r2_score(y_pred,y_test)
r_score
```

```
[14]: 0.7100923694262505
```

When checked the R_Score value is 0.7100

Since the value does not look good trying with SVM

2. SVM

Using rbf as kernel

```
[6]: from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(independent,dependent,test_size=0.30,random_state=0)
```

```
[8]: from sklearn.svm import SVR
#regressor=SVR(kernel="poly",degree=4,gamma="auto",coef0=1,epsilon=0.5,cache_size=500,verbose=True,max_iter=1)
regressor=SVR(kernel="rbf")
regressor.fit(X_train,y_train)
```

```
C:\Anaconda3\Lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was p
se change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
```

```
[8]: ▾ SVR
     ▶ Parameters
```

```
[9]: y_pred = regressor.predict(X_test)
```

```
[10]: from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

```
[10]: -0.08842732776913875
```

The R_Score value is -0.088 which is poor

Now with same kernel and altering other values as below

```
#regressor=SVR(kernel="poly",degree=4,gamma="auto",coef0=1,epsilon=0.5,cache_size=500,verbose=True,max_iter=1)
regressor=SVR(kernel="rbf",degree=4,gamma="auto",coef0=1,epsilon=0.5,cache_size=500,verbose=True,max_iter=1)
regressor.fit(X_train,y_train)
```

```
[LibSVM]
C:\Anaconda3\Lib\site-packages\sklearn\utils\validation.py:1406: DataConversionWarning: A column-vector y was p
se change the shape of y to (n_samples, ), for example using ravel().
  y = column_or_1d(y, warn=True)
C:\Anaconda3\Lib\site-packages\sklearn\svm\_base.py:305: ConvergenceWarning: Solver terminated early (max_iter=
ith StandardScaler or MinMaxScaler.
  warnings.warn(
```

[11]:
```
▾ SVR
▶ Parameters
```

[12]:
```
y_pred = regressor.predict(X_test)
```

[13]:
```
from sklearn.metrics import r2_score
r_score=r2_score(y_test,y_pred)
r_score
```

[13]:
```
-2.146091723438573
```

The R_Score is -2.14 which is again poor. Trying other option of kernel and provided the details in tabular as below –

kernel	degree	gamma	epsilon	cache_size	verbose	max_iter	R_score
rbf	3	scale	0.1	200	False	-1	-0.088
rbf	4	Auto	0.5	500	True	1	-2.14
sigmoid	4	Auto	0.5	500	True	1	-0.0899
poly	3	scale	0.1	200	False	-1	-0.064

Since the R_score is low going to try with other models

3. Decision Tree

criterion	*splitter*	*max_depth*	*min_samples_split*	*min_samples_leaf*	*min_weight_fraction_leaf*	*max_features*	*random_state*	max_leaf_nodes	min_impurity_decrease	ccp_alpha	R_score
squared_error	*random*	*None*	2	1	0.0	None	None	None	0.0	0.0	0.6318
friedman_mse	*best*	*None*	2	1	0.0	None	None	None	0.0	0.0	0.7056
friedman_mse	***random***	***None***	**2**	**1**	**0.0**	**None**	**None**	**None**	**0.0**	**0.0**	**0.7624**

absolute_error	best	None	2	1	0.0	None	None	None	0.0	0.0	0.6666
absolute_error	*random*	*None*	2	1	0.0	None	None	None	0.0	0.0	0.6835
poisson	best	*None*	2	1	0.0	None	None	None	0.0	0.0	0.7229
poisson	random	*None*	2	1	0.0	None	None	None	0.0	0.0	0.6747

As in the above procedure the highest R score is **0.7624,** need to try Random Forest now.

4. **Random Forest**

n_estimators	**criterion**	**max_features**	**bootstrap**	**min_samples_split**	**min_samples_leaf**	**R_score**
100	*squared_error*	1.0	True	2	1	0.8538
100	*absolute_error*	1.0	True	2	1	0.8520
100	*friedman_mse*	1.0	True	2	1	0.8540
100	poisson	1.0	True	2	1	0.8526

The highlighted one looks better compared to other ones above – **0.8540**
But this is low compared to decision tree **R_score. (0.8540)**

Among the above all this seems to the highest value so proceeding with deploying the same.